DEMO WALKTHROUGH

AIOps Ansible Software Factory

One preparation. Two independent RHEL applications. Human gates and verified recovery.

	1 Baseline	2 Trigger	3 Diagnose	4 Author	5 Test	6 Review	7 Deploy	8 Verify
WEB APP SELinux outage demo.webapp RHEL host A AWS EC2	RHEL HOST A **Web app ready** nginx serves /web SELinux Permissive	OPERATOR / AAP **Introduce fault** Run the SELinux Enforcing job on the web host.	EDA / AO / AAP **Detect + diagnose** Probe fails → alert EDA → AO audit-log RCA Forgejo issue → EDA	DEVELOPER HUB / AGENT **Author the fix** Prepare a branch. Sandbox agent adds persistent SELinux labels and opens a PR.	INDEPENDENT OPERATOR **Prove the PR** Test the exact SHA. Lint, build, Molecule. RHEL 9 + 10 full lifecycle and cleanup.	HUMAN GATE **Review + merge** Check the diff and independent evidence. Merge the tested SHA in Forgejo.	OPERATOR / AAP **Recover** Launch one recovery job. AAP deploys the reviewed collection to the web host.	RUNNING WEB APP **Show recovery** SELinux Enforcing. HTTP 200. Probe healthy. Outage alert resolved.
PYTHON APP Invoice CVE demo.pythonapp RHEL host B AWS EC2	RHEL HOST B **Invoice baseline** Normal PDF works. Regression PDF times out at two seconds. SELinux Enforcing.	OPERATOR / AO **Run detection** Start the invoice detection workflow. The service remains available.	AAP / LIGHTWELL / EDA **Find the CVE** Observe the package and invoice behavior. Discover approved fix. Forgejo issue → EDA.	DEVELOPER HUB / AGENT **Update package** Prepare a branch. Agent updates the approved release lock and adds tests → PR.	AO / TRUSTED AAP **Validate exact SHA** Source-path preflight. Candidate + acceptance. RHEL 9 + 10; rollback and cleanup. Publish required check.	HUMAN GATE **Review + merge** Required check passes. Human reviews + merges. AO confirms the same SHA is in main.	HUMAN AAP GATE **Approve production** approved AO / AAP **Deploy pinned release**	AO / RUNNING APP **Show the fix** Prove the deployment. Both unchanged PDFs process successfully. Exact wheel + SHA. Reconcile the incident.

SHARED MANAGEMENT PLANE ON OPENSHIFT

Homepage + SSO • AO • AAP + EDA • Developer Hub • Forgejo • Omnigent + Agent Sandbox • GitOps

WALKTHROUGH NOTE

Follow the numbered columns

Show the baseline, trigger the incident, follow the issue and agent, inspect the PR and tests, approve, then prove recovery.

WALKTHROUGH NOTE

People keep the release gates

Both paths require human review and merge. Python also requires AAP production approval. Agents cannot merge or deploy.

WALKTHROUGH NOTE

Real machines, separate responsibilities

Two production RHEL EC2 hosts. Disposable RHEL 9 + 10 EC2 test guests. Management services and agent sandboxes run on OpenShift.

DEMO WALKTHROUGH

Python invoice CVE — continuous AO workflow

CVE-2023-36464 • approved Lightwell PyPDF2 update • independent acceptance • separate production approval

INCIDENT → AUTHORING

01 OPERATOR / AO

Trigger detection

Start invoice-security-detect.
Show the healthy normal invoice and the regression timeout on the baseline.

02 AAP + LIGHTWELL

Collect trusted findings

Read the installed dependency.
Run business + regression checks.
Discover compatible Lightwell wheels using approved advisory policy.

03 FORGEJO → EDA

Open the incident

AAP creates a deduplicated ticket.
The new-issue webhook reaches EDA and starts invoice-security-remediation in AO.

04 AO + DEVELOPER HUB

Prepare contribution

Validate the repository and issue.
Create feature/issue-N.
Only a newly claimed incident starts a new authoring session.

05 AGENT SANDBOX

Author + test the update

Omnigent runs the configured model.
Update the approved package lock and regression tests.
Test on disposable RHEL 9 + 10 EC2 guests.

06 FORGEJO / AO

Publish the candidate

The agent opens a PR.
AO discovers its exact head SHA and publishes a pending required validation check.

Continue to the tested release below

EVIDENCE → REVIEWED RECOVERY

07 TRUSTED AAP

Independently validate

Preflight source-path contract.
Lint, build, candidate tests.
Separate trusted RHEL 9 + 10 acceptance: regression, rollback, idempotence and cleanup.

08 HUMAN GATE

Review + merge

Inspect the diff and independent evidence.
Required check must pass.
Merge that exact tested SHA.
AO observes the merge.

09 AO / AAP

Configure exact release

Confirm the tested SHA is in main.
Bind source, wheel, profile and execution image to an immutable production workflow.

10 HUMAN AAP GATE

Approve production

Approve the release-specific workflow in AAP.
A rejection or timeout cannot deploy the application.

11 AAP → RHEL HOST B

Deploy approved release

Run the tested collection commit.
Verify the pinned Lightwell artifact and provenance.
Activate the immutable release on the Python host.

12 AO / AAP / PORTAL

Verify + reconcile

Prove approval and deployment.
Check running SHA + wheel.
Process the identical normal and regression PDFs.
Reconcile the incident.

failed

closed / changed SHA

denied / timeout

FAIL CLOSED — KEEP THE EVIDENCE

Failed validation blocks merge. Changed source or denied approval blocks deployment. Failed verification leaves the incident unresolved.

SHOW DURING THE WALKTHROUGH

Invoice portal → AO detection and remediation → Developer Hub / agent → Forgejo PR + required check → AAP approval → same PDFs succeed.

Trusted acceptance lives in demojam-ansible, outside candidate write access. Test guests: RHEL 9 + 10 on EC2. Inference uses the configured provider.

DEMO WALKTHROUGH

Nginx SELinux outage — diagnosed and reviewed recovery

A separate RHEL host • real audit evidence • persistent file labels • independent operator validation

INCIDENT → AUTHORING

01 OPERATOR / AAP
Introduce the outage
Run webapp_selinux_enable on webapps only.
Switch the web host to Enforcing.
The Python host is unaffected.

→

02 MONITORING
Observe HTTP failure
The blackbox probe returns 403.
WebappDown fires after one minute.
Alertmanager delivers the alert to EDA.

→

03 EDA → AO
Start investigation
The alert listener invokes the AO root-cause workflow.
Keep the original alert context through diagnosis.

→

04 AO + AAP
Read live audit logs
AAP collects SELinux audit evidence from the web host.
AO uses the configured model to identify the denial and root cause.

→

05 AAP → FORGEJO
Open the outage issue
AAP creates the remediation ticket with the alert and root-cause evidence in the web collection repository.

→

06 FORGEJO → EDA → AO
Dispatch authoring
The new-issue webhook reaches EDA.
EDA starts omnigent-remediation in AO for this outage issue.

Continue to the tested release below

EVIDENCE → REVIEWED RECOVERY

07 DEVELOPER HUB
Prepare the branch
Validate the incident repository.
Create feature/issue-N.
AO requests the Omnigent authoring session.

→

08 AGENT SANDBOX
Author fix + open PR
Add persistent httpd_sys_content_t labels and relabel /web.
Test the Ansible change.
Publish a PR; leave SELinux Enforcing.

→

09 INDEPENDENT OPERATOR
Test exact published SHA
Use an isolated source export in the existing sandbox.
Lint, build and full Molecule lifecycle on RHEL 9 + 10.
Verify Enforcing + HTTP; destroy guests.

failed tests ↓

→

10 HUMAN GATE
Review + merge
Inspect the code and independent evidence.
Recheck the base and tested head.
Merge with the exact SHA guard.

→

11 OPERATOR / AAP
Launch one recovery
Run webapp nginx once.
AAP installs the reviewed collection on the web host.
No permissive setup after recovery.

failed recovery ↓

→

12 WEB APP / MONITORING
Prove recovery
Verify SELinux Enforcing.
Require HTTP 200 and the correct page.
Confirm the probe is healthy and WebappDown resolves normally.

FAIL CLOSED — KEEP THE EVIDENCE

A failed candidate check leaves the PR open. A failed recovery is recorded as failed. Preserve the evidence and keep the agent branch untouched.

SHOW DURING THE WALKTHROUGH

Web app and firing alert → AO audit-log diagnosis → Forgejo issue → agent / PR → independent RHEL tests → human merge → AAP recovery → healthy probe.

The operator independently tests and launches recovery on this path. Both application and test guests are EC2; OpenShift hosts the management plane.